



Python Function

— VISUALLY EXPLAINED —



www.tech365.ng



08029704730

1. WHAT IS A FUNCTION?

A function is a block of reusable code that performs a specific task.

Think of it as a recipe.



2. DEFINE A FUNCTION

Use the `def` keyword followed by a name, parentheses `()`, and a colon `:`

```
def greet(name):  
    print("Hello, " + name +  
          "!!")
```

↑ keyword ↑ name ↑ parameter

3. CALL A FUNCTION

Call (invoke) the function using its name and pass the required arguments.

```
greet("John"):
```

Output:
Hello, John!

4. PARAMETERS & ARGUMENTS

Parameters are variables in the function definition.
Arguments are the actual values passed while calling.

```
def add(a, b):  
    return a + b
```

```
result = add(5, 3)
```

↑ parameters ↑ arguments

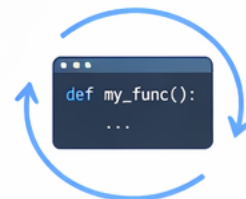
5. RETURN VALUE

Use `return` to send a result back to the caller.

```
def square(x):  
    return x * x  
  
value = square(4)  
print(value) # 16
```

6. REUSABILITY

Write once, use many times.
Functions make code organized, readable and easy to maintain.

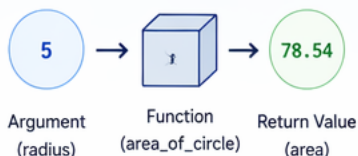


EXAMPLE: CALCULATE AREA OF A CIRCLE

```
import math  
  
def area_of_circle(radius):  
    return math.pi * radius * radius  
  
r = 5  
area = area_of_circle(r)  
print(f"Area: {area:.2f}")
```

Output: Area: 78.54

How it works:



7. TYPES OF FUNCTIONS

a) Built-in Functions

Predefined in Python.

```
print() len() type()  
input() max() min()
```

Example:

```
print(len("Hello"))  
# Output: 5
```

b) User-defined Functions

Created by the programmer.

```
def welcome():  
    print("Welcome!")
```

```
welcome()  
# Output: Welcome!
```

c) Functions with Arguments

Accept inputs (arguments).

```
def add(x, y):  
    return x + y
```

```
add(4, 6) # 10
```

d) Functions with Default Arguments

Have default values.

```
def greet(name="Guest"):  
    print(f"Hi, {name}!")
```

```
greet() # Hi, Guest!  
greet("Alice") # Hi, Alice!
```



Use meaningful names

Choose descriptive function names.



Keep it small

Do one thing and do it well.



Use docstrings

Explain what your function does.

```
def add(a, b):  
    """Return the sum  
    of two numbers."""  
    return a + b
```



Avoid repetition

Reuse your function instead of writing the same code.



Test your function

Test with different inputs and edge cases.



QUICK TAKEAWAY

- ✓ A function is reusable code for a specific task.
- ✓ Define it using `def`, call it by name.
- ✓ It can take inputs (arguments) and return outputs.
- ✓ Functions help make your code clean, modular and easier to maintain.



www.tech365.ng



08029704730



Python Modules & Packages

— VISUALLY EXPLAINED —

www.tech365.ng

08029704730

1. WHAT ARE MODULES?

A module is a single Python file containing Python code (functions, classes, variables, etc.). It can be imported and used in other programs.

Example: my_module.py

```
# my_module.py
def greet(name):
    return f"Hello, {name}!"

PI = 3.14159
```

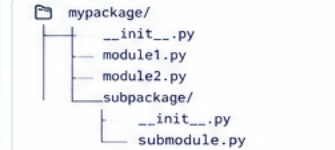
- ✓ Helps in code organization
- ✓ Promotes reusability
- ✓ Improves maintainability

2. WHAT ARE PACKAGES?

A package is a collection of modules and subpackages arranged in directories.

A directory becomes a package when it contains a file named `__init__.py` (can be empty).

Example package structure:



Used to organize related modules in a structured way.

3. IMPORTING MODULES

Import a module and access its contents using dot notation.

```
# my_module.py
PI = 3.14159
def area(r):
    return PI * r * r

# main.py
import my_module

print(my_module.PI)
print(my_module.area(5)) # 78.53975
```

★ **Tip:** Use meaningful module names. Avoid naming your file same as built-in modules.

4. IMPORTING SPECIFIC ITEMS

Import specific function, class or variable from a module.

```
from my_module import area, PI
print(PI)
print(area(5))
```

Import with an alias:

```
import my_module as mm
print(mm.PI)
print(mm.area(5))
```

Import specific item with alias:

```
from my_module import area as calc_area
print(calc_area(5))
```

5. TYPES OF IMPORTS

Type	Example	Description
Import module	<code>import math</code>	Imports entire module
Import specific item	<code>from math import sqrt</code>	Imports specific item
Import multiple items	<code>from math import sqrt, pi</code>	Imports multiple items
Import all items (Not recommended)	<code>from math import *</code>	Imports all names from module
Import with alias	<code>import numpy as np</code>	Imports module with alias

Avoid using 'from module import *'. It can cause name conflicts and reduces code clarity.

6. USING MODULES

Create a module and use it in another file.

```
math_utils.py + main.py

def add(a, b):
    return a + b

def sub(a, b):
    return a - b

import math_utils

print(math_utils.add(10, 5)) # 15
print(math_utils.sub(10, 5)) # 5
```

Place modules in the same directory or in a directory included in `PYTHONPATH`.

7. USING PACKAGES

Import modules from a package.

```
# Project structure
mypackage/
├── __init__.py
├── module1.py
├── module2.py
└── subpackage/
    ├── __init__.py
    └── submodule.py

# main.py
from mypackage import module1

print(module1.some_function())
```

★ You can also import submodules:
`from mypackage.subpackage import submodule`

8. RELATIVE IMPORTS (IN PACKAGES)

Use relative imports within a package.

Package structure:

```
graph TD
    mypackage["mypackage/"]
    mypackage --- init1["__init__.py"]
    mypackage --- module1["module1.py"]
    mypackage --- module2["module2.py"]
    mypackage --- subpackage["subpackage/"]
    subpackage --- init2["__init__.py"]
    subpackage --- module2_sub["module2.py"]
```

inside module2.py

```
from .. import module1 # go up one level
print(module1.some_function())
```

. -> current package
.. -> parent package ... -> grandparent package

9. BUILT-IN MODULES (EXAMPLES)

Python provides many built-in modules.

Module	Purpose	Example
math	Mathematical functions	<code>import math</code>
random	Random numbers	<code>import random</code>
datetime	Date and time	<code>import datetime</code>
os	Operating system interactions	<code>import os</code>
sys	System-specific parameters	<code>import sys</code>
json	JSON encoding/decoding	<code>import json</code>

10. `__name__` VARIABLE

Every Python module has a special variable `__name__`.

```
# my_module.py
if __name__ == "__main__":
    print("This file is run directly")
else:
    print("This file is imported")
```

When a file is run directly, `__name__` is `"__main__"`. When imported in another file, it is set to the module's name.

★ Useful to run code only when the module is executed directly.

11. MODULE VS PACKAGE

Module	Package
A single .py file	A directory of modules
Has .py extension	Contains <code>__init__.py</code> file
Cannot contain submodules	Can contain submodules and subpackages
Example: <code>math.py</code>	Example: <code>mypackage/</code>

VS

12. ADVANTAGES

- ✓ Better code organization
- ✓ Reusability of code
- ✓ Easier maintenance and debugging
- ✓ Avoids code duplication
- ✓ Supports large scale projects

13. BEST PRACTICES

- ✓ Use meaningful module and package names
- ✓ Keep modules focused (single responsibility)
- ✓ Use `__init__.py` to initialize package (optional code)
- ✓ Avoid circular imports
- ✓ Follow PEP 8 naming conventions
- ✓ Document your modules (docstrings)

14. COMMON USE CASES

- Break large programs into smaller files
- Share code across multiple projects
- Group related functionality (packages)
- Build reusable libraries
- Organize web apps, APIs, data pipelines

15. EXAMPLE: REAL PROJECT STRUCTURE

Project: myproject/



```
# main.py
from utils import helpers
from data.loader import load_data

helpers.greet("Alice")
data = load_data()
print(data)
```



QUICK SUMMARY

- ✓ Module = single Python file (.py)
- ✓ Package = collection of modules (with `__init__.py`)
- ✓ Use `import` to reuse code
- ✓ Organize code for better readability, reuse and scale



HELPFUL TIP

Use `dir(module)` to see all names in a module.
Use `help(module)` for documentation.



REMEMBER

Good structure today,
maintainable project tomorrow!



TRY IT YOURSELF!

- Create a module with some functions and import it.
- Create a package with multiple modules.
- Use relative imports inside the package.



USEFUL COMMANDS (PYTHON SHELL)

```
> import module # import module
> module.__name__ # module name
> dir(module) # list names in module
> help(module) # show help
```



Working with Files in Python

— VISUALLY EXPLAINED —

www.tech365.ng

08029704730

1. FILES IN PYTHON

A file is a named location on disk used to store data permanently. Python provides built-in functions to create, read, update and delete files.

Example file (sample.txt):

```
Hello Python!
Files are easy to handle.
Happy Coding!
```

- ✔ Data persists even after program ends
- ✔ Types: Text files, Binary files

2. OPENING A FILE

Use the `open()` function to open a file.

```
open(filename, mode, encoding)
```

Mode	Description
'r'	Read (default)
'w'	Write (creates new or truncates)
'a'	Append (adds to end of file)
'x'	Create (fails if file exists)
'b'	Binary mode
't'	Text mode (default)
'+'	Read and write



Always close the file using `close()` or use `'with'` statement (recommended).

3. READING DATA

Different ways to read from a file.

<code>read(size=-1)</code>	Read entire file (size in characters)
<code>readline(size=-1)</code>	Read one line
<code>readlines()</code>	Read all lines into a list
<code>for line in file</code>	Iterate line by line

```
# Example
with open('sample.txt', 'r', encoding='utf-8') as f:
    data = f.read()
    print(data)
    f.seek(0)          # move cursor to start
    line = f.readline()
    print(line)
```

4. WRITING DATA

Write data to a file.

```
# Overwrite (mode 'w')
with open('out.txt', 'w', encoding='utf-8') as f:
    f.write('Hello Python!\n')
    f.write('Writing to a file.\n')
```

```
# Append (mode 'a')
with open('out.txt', 'a', encoding='utf-8') as f:
    f.write('Appending this line.\n')
```



'w' overwrites the file if it exists.
'a' adds new content at the end.

5. APPENDING DATA

Add new data to the end of a file.

```
with open('log.txt', 'a', encoding='utf-8') as f:
    f.write('New entry 1\n')
    f.write('New entry 2\n')
```



Append mode ('a') creates the file if it does not exist.

6. WITH STATEMENT (RECOMMENDED)

Automatically handles closing the file.

```
with open('sample.txt', 'r', encoding='utf-8') as f:
    for line in f:
        print(line.strip())
```

- ✔ File is properly closed even if an error occurs.
- ✔ Best practice for working with files.

7. FILE METHODS

Commonly used methods and their purpose.

Method	Description
<code>close()</code>	Close the file
<code>tell()</code>	Return current cursor position
<code>seek(offset, whence=0)</code>	Change cursor position <code>whence=0</code>: start (default) <code>whence=1</code>: current position <code>whence=2</code>: end
<code>truncate(size=None)</code>	Resize file (default: current position)
<code>flush()</code>	Flush internal buffer to disk
<code>writable()</code>	Check if file is opened in write mode
<code>readable()</code>	Check if file is opened in read mode

8. BINARY FILES

Work with non-text data (images, videos, etc.).

```
# Writing binary data
with open('image.png', 'wb') as f:
    f.write(b'\xB9PNG\r\n...')

# Reading binary data
with open('image.png', 'rb') as f:
    data = f.read()
    print(len(data), 'bytes read')
```

★ Use 'b' in mode for binary files (rb, wb, ab).

9. CHECKING FILE PROPERTIES

Use functions from `os` module.

```
import os
os.path.exists('sample.txt')  # True/False
os.path.isfile('sample.txt')  # True/False
os.path.getsize('sample.txt') # size in bytes
```



Useful to check if a file exists before performing operations.

10. CLOSING A FILE

Close the file when work is done.

```
f = open('sample.txt', 'r')
print(f.read())
f.close()  # Important!
```



Always close files that are not opened with `'with'`. It frees system resources.

11. EXAMPLE: FULL WORKFLOW

Read, write and append to a file.

```
# Write
with open('demo.txt', 'w') as f:
    f.write('Line 1\n')

# Append
with open('demo.txt', 'a') as f:
    f.write('Line 2\n')

# Read
with open('demo.txt', 'r') as f:
    content = f.read()
    print(content)
```

12. EXAMPLE: LINE BY LINE PROCESSING

Process a file line by line.

```
with open('data.txt', 'r') as f:
    for i, line in enumerate(f, start=1):
        print(f'{i}: {line.strip()}')
```



`enumerate()` gives line numbers automatically.

13. COMMON FILE MODES QUICK REFERENCE

Mode	Meaning	Creates File	Truncates	Pointer Position
'r'	Read	No	No	Start
'w'	Write	Yes	Yes	Start
'a'	Append	Yes	No	End
'x'	Create (new)	Yes*	Yes	Start
'r+'	Read & Write	No	No	Start
'w+'	Write & Read	Yes	Yes	Start
'a+'	Append & Read	Yes	No	End

* Fails if file exists

14. COMMON EXCEPTIONS

Handle file related errors.

```
FileNotFoundError  # File not found
PermissionError    # No permission
IsADirectoryError  # Path is a directory
IOError / OSError  # General I/O error
```

```
try:
    with open('missing.txt', 'r') as f:
        print(f.read())
except FileNotFoundError:
    print('File not found!')
```

15. FILE OPERATIONS SUMMARY

- ✔ Use `open()` to open files.
- ✔ Use `'with'` statement for safety.
- ✔ Read using `read()`, `readline()`, `readlines()`.
- ✔ Write using `write()`.
- ✔ Append using mode `'a'`.
- ✔ Close files properly.
- ✔ Check file existence and size when needed.



BEST PRACTICES

- Use `'with'` statement.
- Close files when not using `'with'`.
- Use meaningful file names.
- Handle exceptions properly.
- Choose the right mode for the task.



COMMON USE CASES

- Reading configuration files
- Writing logs
- Saving user data
- Processing text files
- Working with CSV, JSON, etc.



HELPFUL TIP

File cursor (pointer) keeps track of the current position in the file. Use `tell()` and `seek()` to control it.



REMEMBER

- 'w' overwrites, 'a' appends.
- Always specify encoding for text files.
- Binary files handle bytes, not strings.



TRY IT YOURSELF!

Create a file, write your name and age. Read it back and print the content. Then append a new line and read again.



www.tech365.ng



08029704730